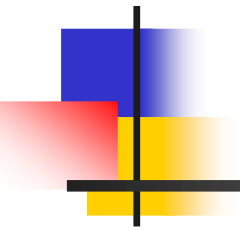
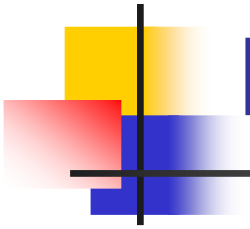


Linux MultiCore Programming

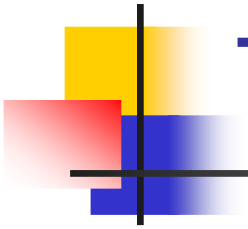
Enyi Tang

Software Institute of
Nanjing University



Linux Process

- The entry and ext of C programs
- System call “exec”
- System call “fork”



The entry of C programs

- crt0.o
  - cc/ld
- main function
  - function prototype：

    int main(int argc, char *argv[]);

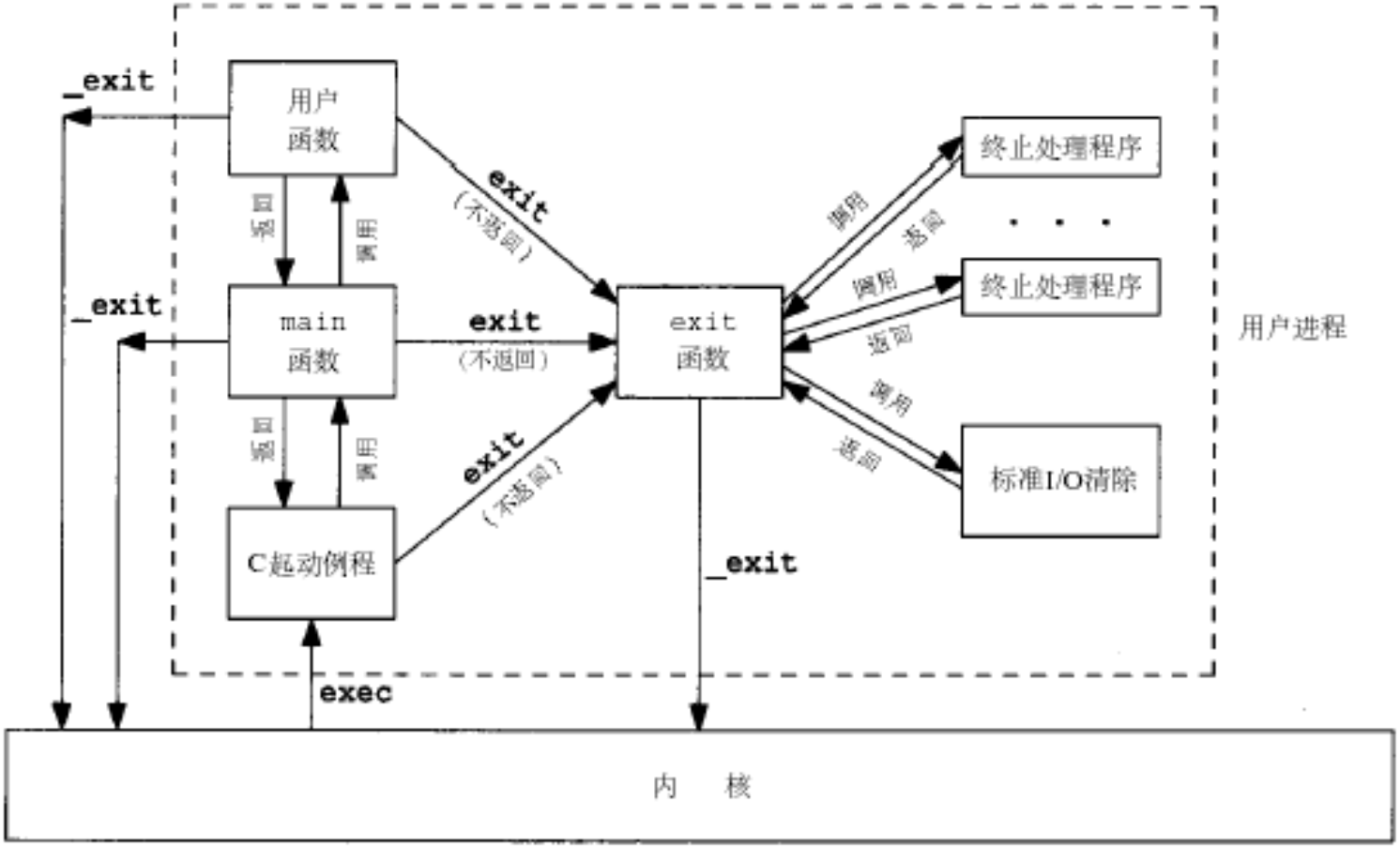
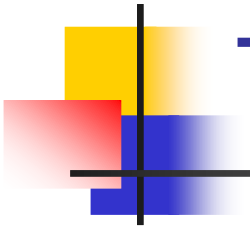


图7-1　一个C程序是如何起动和终止的



The termination of a process

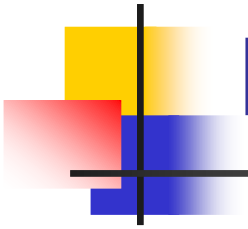
- Five ways of terminating a process
  - Normal termination
    - Return from “main” function
    - Call “exit” function
    - Call “_exit” function
    - Process with threads：last thread exit
  - Abnormal termination
    - Call “abort” function
    - Terminated by a signal
    - Last thread is cancelled

exit & _exit functions

- Function prototype:

```
#include <stdlib.h>
void exit(int status);
#include <unistd.h>
void _exit(int status);
```

- Exit status
  - How about the status without explicit call?
- Difference
  - _exit is corresponding to a system call, while “exit” is a library function.
  - _exit terminate a process immediately.



Exit handler

- atexit function□
  - Register a function to be called at normal program termination.
  - Prototype:

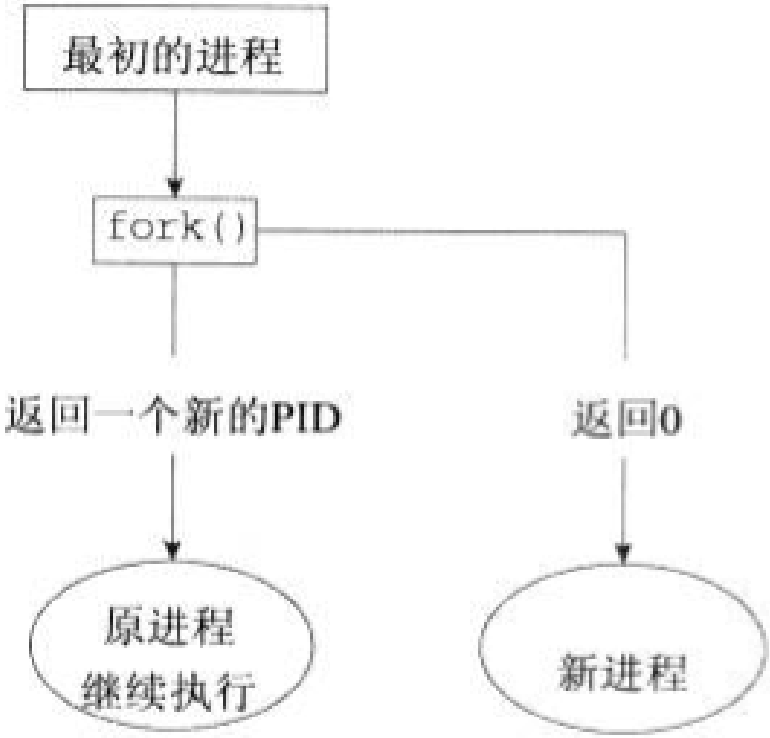
```
#include <stdlib.h>
int atexit(void (*function)(void));□
```

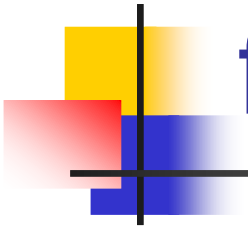
exec 系列函数的使用

```
#include <unistd.h>

int execl(const char *path, const char *arg0, ..., (char *)0);
int execlp(const char *file, const char *arg0, ..., (char *)0);
int execle(const char *path, const char *arg0, ..., (char *)0, char *const
envp[]);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execve(const char *path, char *const argv[], char *const envp[])
```

fork 的使用

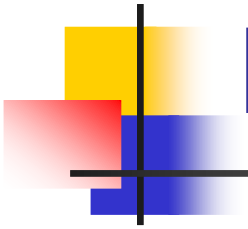
```
#include <sys/types.h>
#include <unistd.h>

pid_t fork(void);
```





fork 的使用

```
if(fork()==0)
    {子进程执行的代码段；}
else
    {父进程执行的代码段；}
```



Process resources

- struct task_struct (进程描述符)
- Other resources
  - Thread Info
  - File Info
  - Signal Info
  - System space stack

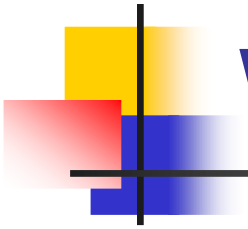
wait & waitpid functions

- Wait for process termination
- Prototype

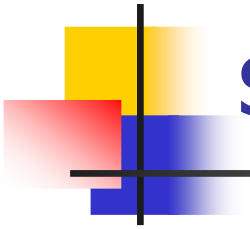
```
#include <sys/types.h>
#include <sys/wait.h>
pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

- 几种可能的结果
  - 阻塞/立即返回/出错

wait vs. waitpid

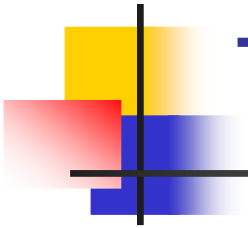


- 指定pid
- 非阻塞
- waitpid的pid参数：
  - Pid ==-1
  - >0
  - ==0
  - <0



signal

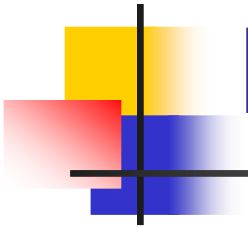
- the concept of signals
- the “signal” function
- send a signal
  - kill, raise
- the “alarm” and “pause” functions
- reliable signal mechanism



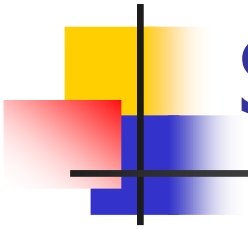
The concept of signals

- Signal
  - Software interrupt
  - Mechanism for handling asynchronous events
  - Having a name (beginning with SIG)
  - Defined as a positive integer (in <signal.h>)
- How to produce a signal
  - 按终端键，硬件异常，kill(2)函数，kill(1)命令，软件条件，...

Produce a signal



- 终端特殊按键（ctrl+c-→SIGINT)
- 硬件异常中断信号(内存错误：SIGSEGV)
- Kill（2）
- Kill（1）
- 当检测到某种软件条件已经发生，并应将其通知到有关进程时（SIGALARM)



Signal handling

- 忽略信号
  - 不能忽略的信号：
    - SIGKILL, SIGSTOP
    - 一些硬件异常信号
- 执行系统默认动作
- 捕捉信号

Signals in Linux/UNIX

名称	说明
SIGABRT	进程异常终止（调用abort函数产生此信号）
SIGALRM	超时（alarm）
SIGFPE	算术运算异常（除以0，浮点溢出等）
SIGHUP	连接断开
SIGILL	非法硬件指令
SIGINT	终端中断符(Clt-C)
SIGKILL	终止（不能被捕捉或忽略）
SIGPIPE	向没有读进程的管道写数据
SIGQUIT	终端退出符(Clt-\)
SIGTERM	终止（由kill命令发出的系统默认终止信号）
SIGUSR1	用户定义信号
SIGUSR2	用户定义信号

Signals in Linux/UNIX (cont'd)

名称	说明
SIGSEGV	无效存储访问（段违例）
SIGCHLD	子进程停止或退出
SIGCONT	使暂停进程继续
SIGSTOP	停止（不能被捕捉或忽略）
SIGTSTP	终端挂起符(Clt-Z)
SIGTTIN	后台进程请求从控制终端读
SIGTTOUT	后台进程请求向控制终端写

The “signal” function

- Installs a new signal handler for the signal with number signum.

```
#include <signal.h>
typedef void (*sighandler_t)(int);
sighandler_t signal(int signum, sighandler_t handler);
```

  (Returned Value: the previous handler if success, SIG_ERR if error)

- The “handler” parameter
  - a user specified function, or
  - SIG_DEF, or
  - SIG_IGN
- 进程创建/程序执行

The ”signal” function (cont’d)

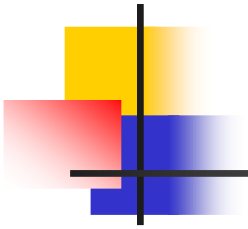
- Program example

```
static void sig_usr(int);
int main(void)
{
    if (signal(SIGUSR1, sig_usr) == SIG_ERR)
        err_sys("can't catch SIGUSR1");
    if (signal(SIGUSR2, sig_usr) == SIG_ERR)
        err_sys("can't catch SIGUSR2");

    for ( ; ; )
        pause();
}
```

Signal信号的可靠性阐述

- 是否每个发生的信号都能捕捉?
- 阻塞信号
- 信号处理的复位机制

```
int    sig_int();          /* my signal handling function */

...
signal(SIGINT, sig_int); /* establish handler */
...

sig_int()
{
    signal(SIGINT, sig_int); /* reestablish handler for next time */
    ...                      /* process the signal ... */
}
```

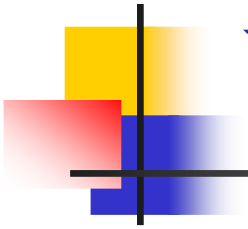


中断系统调用

- 低速系统调用在接受到信号时被中断
  - 在读/写某些类型的文件（管道，终端，网络）
  - 在打开某些类型的设备
  - Pause
  - Wait
  - Ioctl
  - 进程间通讯
  - 返回出错,errno=EINTR

中断系统调用

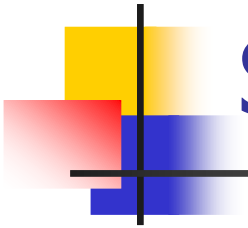
```
again:
    if ((n = read(fd, buf, BUFFSIZE)) < 0) {
        if (errno == EINTR)
            goto again;      /* just an interrupted system call */
        /* handle other errors */
    }
```



可重入函数

- 可以被中断的函数
- 不可重入函数：
  - 系统资源
  - 全局变量
  - 使用静态数据结构
  - 调用malloc或者free
  - 标准IO函数
  - 例子：getpwname(),errno,reenter.c

abort	faccessat	linkat	select	socketpair
accept	fchmod	listen	sem_post	stat
access	fchmodat	lseek	send	symlink
aio_error	fchown	lstat	sendmsg	symlinkat
aio_return	fchownat	mkdir	sendto	tcdrain
aio_suspend	fcntl	mkdirat	setgid	tcflow
alarm	fdatasync	mkfifo	setpgid	tcflush
bind	fexecve	mkfifoat	setsid	tcgetattr
cfgetispeed	fork	mknod	setsockopt	tcgetpgrp
cfgetospeed	fstat	mknodat	setuid	tcsendbreak
cfsetispeed	fstatat	open	shutdown	tcsetattr
cfsetospeed	fsync	openat	sigaction	tcsetpgrp
chdir	ftruncate	pause	sigaddset	time
chmod	futimens	pipe	sigdelset	timer_getoverrun
chown	getegid	poll	sigemptyset	timer_gettime
clock_gettime	geteuid	posix_trace_event	sigfillset	timer_settime
close	getgid	pselect	sigismember	times
connect	getgroups	raise	signal	umask
creat	getpeername	read	sigpause	uname
dup	getpgrp	readlink	sigpending	unlink
dup2	getpid	readlinkat	sigprocmask	unlinkat
execl	getppid	recv	sigqueue	utime
execle	getsockname	recvfrom	sigset	utimensat
execv	getsockopt	recvmsg	sigsuspend	utimes
execve	getuid	rename	sleep	wait
_Exit	kill	renameat	sockatmark	waitpid
_exit	link	rmdir	socket	write



Send a signal

- kill(2): send signal to a process

  #include <sys/types.h>

  #include <signal.h>

  int kill(pid_t pid, int sig);

  (Returned Value: 0 if success, -1 if failure)

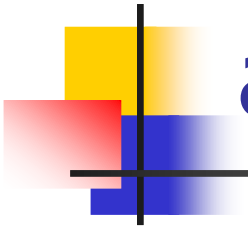
  pid:取值

- raise(3): send a signal to the current process

  #include <signal.h>

  int raise(int sig);

  (Returned Value: 0 if success, -1 if failure)



alarm & pause functions

- alarm: set an alarm clock for delivery of a signal

  #include <unistd.h>

  unsigned int alarm(unsigned int seconds);

  (Returned value: 0, or the number of seconds remaining of previous alarm)

  SIGALRM

- pause: wait for a signal

  #include <unistd.h>

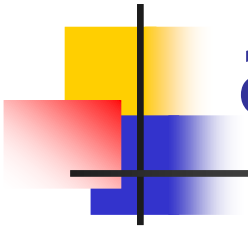
  int pause(void);

  (Returned value: -1, errno is set to be EINTR)

alarm & pause functions (cont'd)

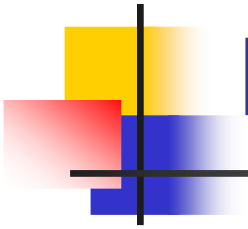
- Program example
  - The implementation of the "sleep" function

```
void sig_alrm(int signo)
{printf("alarm received\n");}
unsigned int sleep1(unsigned int nsecs)  {
    if ( signal(SIGALARM, sig_alrm) == SIG_ERR)
        return(nsecs);
    alarm(nsecs);          /* start the timer */
    pause();       /*next caught signal wakes us up*/
    return(alarm(0) ); /*turn off timer, return unslept time */
}
```



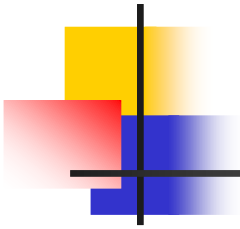
alarm:超时限制

- 对可能阻塞的操作做时间限制
  - 使用signal,被中断的系统调用是否会重启，由具体的操作系统实现决定（BSD/SYS V/LINUX）



Reliable signal mechanism

- Weakness of the signal function
- Signal block
  - signal mask
- Signal set
  - sigset_t data type
- Signal handling functions using signal set
  - sigprocmask, sigaction, sigpending, sigsuspend



signal set operations

```
#include <signal.h>

int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signum);
int sigdelset(sigset_t *set, int signum);
                    (Return value: 0 if success, -1 if error)

int sigismember(const sigset_t *set, int signum);
                    (Return value: 1 if true, 0 if false)
```

sigprocmask function

- 检测或更改(或两者)进程的信号掩码

  #include <signal.h>

  int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);

  (Return Value: 0 is success, -1 if failure)

- 参数“how”决定对信号掩码的操作
  - SIG_BLOCK: 将set中的信号添加到信号掩码(并集)
  - SIG_UNBLOCK: 从信号掩码中去掉set中的信号(差集)
  - SIG_SETMASK: 把信号掩码设置为set中的信号
- 在sigprocmask调用后任何未阻塞并且pending的信号，在函数返回前，至少有一个信号会送达进程
- 例外: SIGKILL, SIGSTOP

sigpending function

- 返回当前未决的信号集

```
#include <signal.h>
int sigpending(sigset_t *set);
```

(Returned Value: 0 is success, -1 if failure)

sigaction function

- 检查或修改(或两者)与指定信号关联的处理动作

```
#include <signal.h>
int sigaction(int signum, const struct sigaction *act, struct
    sigaction *oldact);
                (Returned Value: 0 is success, -1 if failure)
```

- struct sigaction至少包含以下成员:

```
handler_t sa_handler;  /* addr of signal handler, or SIG_IGN,
    or SIG_DEL */
sigset_t sa_mask;    /* additional signals to block */
int sa_flags;          /* signal options */
```

- 可靠的处理方式:永久设置

表10-5　信号处理的选择项标志(sa_flags)

可　选　项	POSIX.1	SVR4	4.3+BSD	说　　明
SA_NOCLDSTOP	•	•	•	若*signo*是SIGCHLD，当一子进程停止时（作业控制），不产生此信号。当一子进程终止时，仍旧产生此信号(但请参阅下面说明的SVR4 SA_NOCLDWAIT可选项)
SA_RESTART		•	•	由此信号中断的系统调用自动再起动（参见 10.5节）
SA_ONSTACK		•	•	若用sigaltstack(2)已说明了一替换栈，则此信号递送给替换栈上的进程
SA_NOCLDWAIT		•		若*signo*是SIGCHLD，则当调用进程的子进程终止时，不创建僵死进程。若调用进程在后面调用 wait，则阻塞到它所有子进程都终止，此时返回 -1，errno设置为ECHILD(见10.7节)
SA_NODEFER		•		当捕捉到此信号时，在执行其信号捕捉函数时，系统不自动阻塞此信号。注意，此种类型的操作对应于早期的不可靠信号
SA_RESETHAND		•		对此信号的处理方式在此信号捕捉函数的入口处复置为SIG_DFL。注意，此种类型的信号对应于早期的不可靠信号
SA_SIGINFO		•		此选项对信号处理程序提供了附加信息。详细情况见10.21节

假如我们想Unblock并等待某个信号

```
/* block SIGINT and save current signal mask */
if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0)
    err_sys("SIG_BLOCK error");

/* critical region of code */

/* reset signal mask, which unblocks SIGINT */
if (sigprocmask(SIG_SETMASK, &oldmask, NULL) < 0)
    err_sys("SIG_SETMASK error");

/* window is open */
pause();  /* wait for signal to occur */

/* continue processing */
```

sigsuspend function

- 用sigmask临时替换信号掩码，在捕捉一个信号或发生终止该进程的信号前，进程挂起。

```
#include <signal.h>
int sigsuspend(const sigset *sigmask);
    (Returned value: -1, errno is set to be EINTR)
```

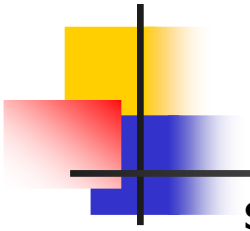
signal function review

- 用sigaction实现signal函数

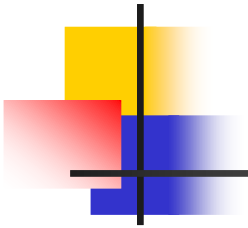
```
sigfunc * signal(int signo, handler_t func) {
    struct sigaction   act, oact;
    act.sa_handler = func;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;
    if (signo == SIGALRM) {
#ifdef      SA_INTERRUPT
        act.sa_flags |= SA_INTERRUPT;     /* SunOS */
#endif
    } else {
#ifdef      SA_RESTART
        act.sa_flags |= SA_RESTART;       /* SVR4, 44BSD */
#endif
    }
    if (sigaction(signo, &act, &oact) < 0)
        return(SIG_ERR);
    return(oact.sa_handler);
}
```

Example: solution of race condition

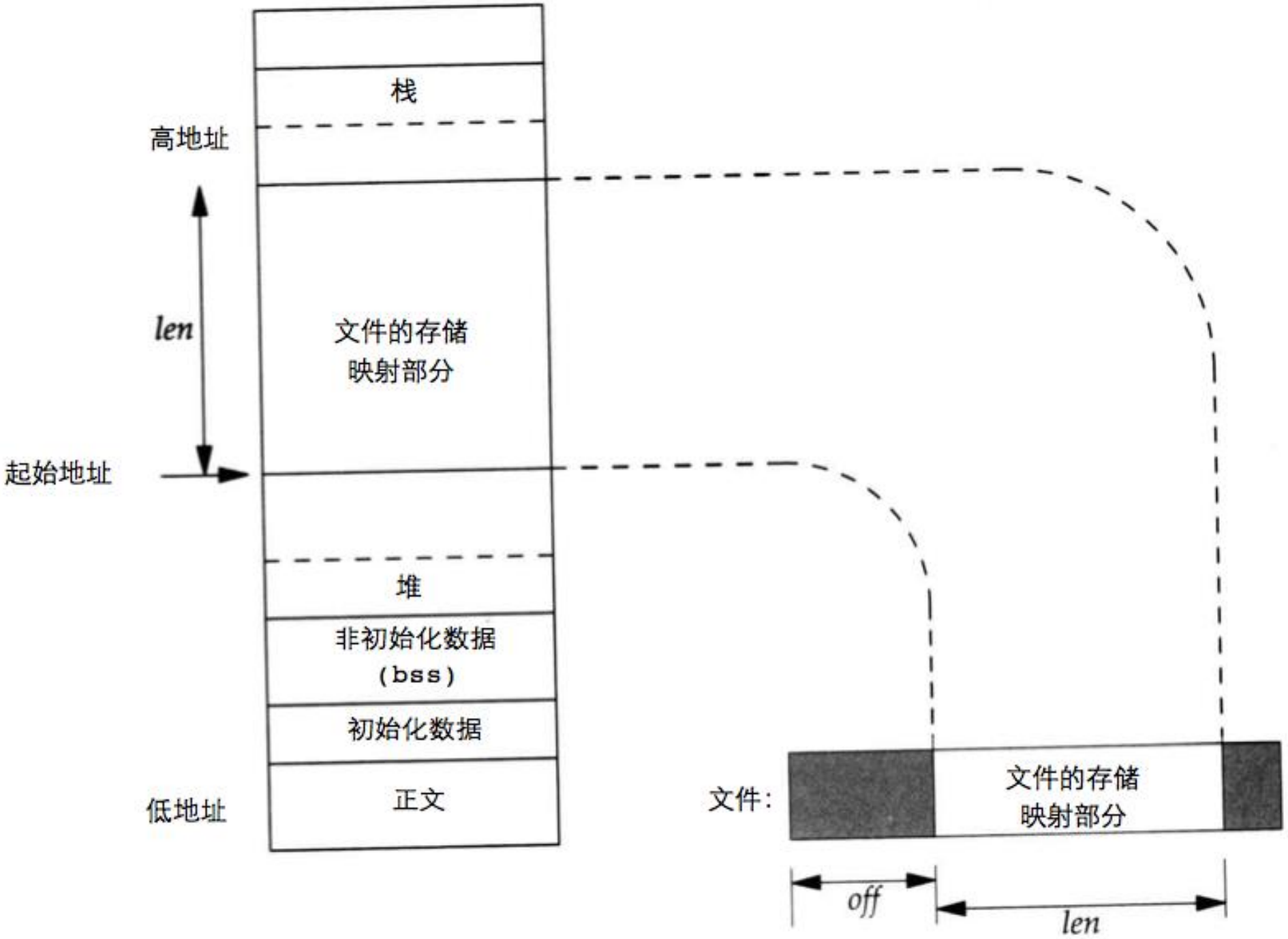
```
int main(void){
    pid_t  pid;
    TELL_WAIT();
    if ( (pid = fork()) < 0)
        err_sys("fork error");
    else if (pid == 0) {
        WAIT_PARENT();          /* parent goes first */
        charatatime("output ccccccccccc from child\n");
    } else {
        charatatime("output pppppppppp from parent\n");
        TELL_CHILD(pid);
    }
    exit(0);
}
```

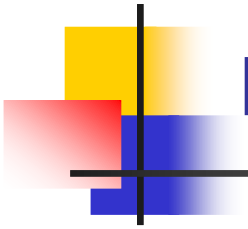


```
static sigset_t      newmask, oldmask, zeromask;

static void sig_usr(int signo) {/* handler for SIGUSR1 */
    sigflag = 1;    return;
}
void TELL_WAIT() {
    if (signal(SIGUSR1, sig_usr) == SIG_ERR)
     err_sys("signal(SIGINT) error");
    sigemptyset(&zeromask);
    sigemptyset(&newmask);
    sigaddset(&newmask, SIGUSR1);
    /* block SIGUSR1, and save current signal mask */
    if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0)
     err_sys("SIG_BLOCK error");
}
```



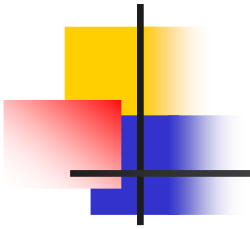
```
void WAIT_PARENT(void) {
    while (sigflag == 0)
     sigsuspend(&zeromask);    /* and wait for parent */

    sigflag = 0;
    /* reset signal mask to original value */
    if (sigprocmask(SIG_SETMASK, &oldmask, NULL) < 0)
     err_sys("SIG_SETMASK error");
}

void TELL_CHILD(pid_t pid) {
    kill(pid, SIGUSR1);             /* tell child we're done */
}
```


栈
高地址
len
文件的存储
映射部分
起始地址
堆
非初始化数据
(bss)
初始化数据
正文
低地址
文件:
文件的存储
映射部分
off
len



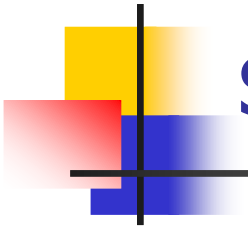
mmap/munmap system calls

- mmap/munmap: map or unmap files or devices into memory
- void* mmap(void* addr,size_t length,
  int prot, int flags,int fd,off_t offset)
  - MAP_SHARED,MAP_ANONYMOUS,MAP_PRIVATE
  - PROT_READ/PROT_WRITE/PROT_EXEC/PROT_NONE
- int munmap(void* addr, size_t length)



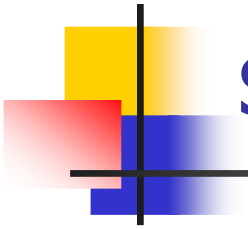
Shared memory

- 共享内存
  - 共享内存是内核为进程创建的一个特殊内存段，它可连接(attach)到自己的地址空间，也可以连接到其它进程的地址空间
  - 最快的进程间通信方式
  - 不提供任何同步功能



shared memory system calls

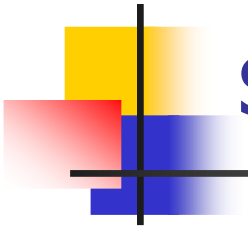
```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
int shmget(key_t key, int size, int flag);
```



shared memory system calls

int shmget(key_t key, int size, int flag);

```
shmid = shmget((key_t)1234, sizeof(struct shared_use_st),
0666 | IPC_CREAT);

if (shmid == -1) {
    fprintf(stderr, "shmget failed\n");
    exit(EXIT_FAILURE);
}
```

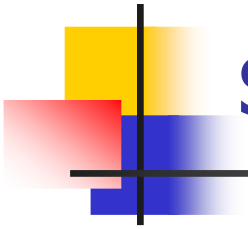


shared memory system calls

void *shmat(int shmid, void *addr, int flag);

- addr:
- flag: SHM_RND, SHM_RDONLY

```
shared_memory = shmat(shmid, (void *)0, 0);
    if (shared_memory == (void *)-1) {
        fprintf(stderr, "shmat failed\n");
        exit(EXIT_FAILURE);
    }
```

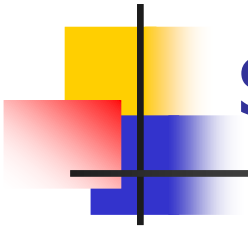


shared memory system calls

int shmdt(void *addr);

- addr:

```
if (shmdt(shared_memory) == -1) {
        fprintf(stderr, "shmdt failed\n");
        exit(EXIT_FAILURE);
    }
```



shared memory system calls

int shmctl(int shmid, int cmd, struct shmid_ds *buf);

- cmd: IPC_STAT, IPC_SET, IPC_RMID

```
if (shmctl(shmid, IPC_RMID, 0) == -1) {
        fprintf(stderr, "shmctl(IPC_RMID) failed\n");
        exit(EXIT_FAILURE);
    }
```